

Sistema Inteligente de Soporte Hospitalario

LaSalle Health Center

MEMORIA TECNICA

Plataforma containerizada de IA y Big Data
para clasificacion radiologica y triaje clinico

Equipo de desarrollo

Oriol Fontcuberta

Unai Canet

Albert Garrido

Asignatura: Sistemes d'Aprenentatge Automatic / Big Data

Fecha de entrega: 2026-05-20

0. Indice

1. Resumen ejecutivo
2. Descripcion del problema
3. Datos
4. Arquitectura del sistema
5. Modelos de Inteligencia Artificial
6. Pipeline de Big Data
7. Backend (API REST)
8. Dashboard
9. Automatizaciones
10. Integraciones
11. Calidad, testing y CI/CD
12. Justificaciones tecnicas y alternativas
13. Diario de desarrollo con IA
14. Reflexion critica
15. Consideraciones eticas y legales
16. Conclusiones
 - A. Anexo: estructura del repositorio

1. Resumen ejecutivo

1.1 Vision general del sistema

Este documento describe la plataforma desarrollada para el hospital ficticio **LaSalle Health Center**, una solución integral de soporte clínico que combina técnicas de inteligencia artificial y big data en un entorno containerizado. El sistema cubre el ciclo completo: ingesta de imágenes radiológicas y datos tabulares, validación de calidad, clasificación automática (radiografía y triaje), persistencia dual (PostgreSQL + MinIO), generación de informes y visualización operativa a través de un dashboard web con autenticación.

La arquitectura se ha diseñado siguiendo principios de separación estricta de responsabilidades (Layered Architecture y Domain-Driven Design adaptados a Python), garantizando transaccionalidad ACID en todas las operaciones compuestas y exposición de métricas operativas en formato JSON consumibles por el dashboard. El despliegue se realiza con **un único comando** (*docker compose up -d*) y cubre nueve servicios containerizados coordinados a través de una red Docker común.

1.2 Tecnologías clave

- **Backend:** FastAPI 0.11x con Pydantic, arquitectura en capas (api / services / repositories / db / storage).
- **Dashboard:** Flask con app factory, blueprints, autenticación local con hashes Werkzeug y sesiones firmadas.
- **Base de datos relacional:** PostgreSQL 16.13 (alpine) con connection pool psycopg 2-10.
- **Almacenamiento de objetos:** MinIO compatible con S3 (boto3).
- **Procesamiento distribuido:** Apache Spark 3.5.8 con un master y un worker.
- **Modelos IA:** Random Forest calibrado (traje) y ResNet18 (clasificación radiológica).
- **Orquestación:** Docker Compose con perfiles (*pipeline*, *training*, *triage-training*).
- **Tests y CI:** pytest con dobles que respetan contratos reales y GitHub Actions.

2. Descripcion del problema

2.1 Contexto del hospital LaSalle Health Center

LaSalle Health Center es una organizacion sanitaria de tamano medio en proceso de transformacion digital. Genera diariamente grandes volumenes de datos clinicos y operativos (historias clinicas, registros de pacientes, pruebas diagnosticas, logs de sistemas) que hasta la fecha no se aprovechan sistematicamente para extraer conocimiento, detectar patrones ni automatizar tareas repetitivas.

La direccion del hospital ha contratado al equipo, en su rol de consultora tecnologica especializada en Inteligencia Artificial y Big Data, para disenar e implementar una plataforma que aporte valor real en un entorno sanitario, respetando las restricciones eticas, legales y operativas inherentes al sector.

2.2 Necesidades detectadas

- Extraer conocimiento util de grandes volumenes de datos heterogeneos.
- Detectar patrones clinicos relevantes en imagenes medicas (radiografias de torax).
- Automatizar procesos internos: validacion de datos, generacion de informes, alertas.
- Apoyar la toma de decisiones medicas y operativas con metricas y resultados interpretables.
- Disponer de una infraestructura reproducible, desplegable con un unico comando y portable a cualquier maquina.

2.3 Objetivos del proyecto

Los objetivos concretos planteados son los siguientes:

- Implementar un sistema de **clasificacion triple de radiografias de torax** en las categorias *Sana*, *Neumonia* y *COVID-19*.
- Disponer de un mecanismo de **triaje clinico automatizado** que asigne nivel de riesgo y prioridad a partir de sintomas y signos vitales.
- Disenar un **pipeline Big Data** con Spark que ingiera, valide, transforme y registre lotes de estudios.
- Combinar dos tipos de almacenamiento: **PostgreSQL** para datos estructurados y **MinIO** para datos no estructurados (imagenes, informes JSON).
- Exponer una **API REST** tipada para consumo desde el dashboard y desde clientes externos.
- Ofrecer una **interfaz web** con autentificacion para personal sanitario.
- Implementar **automatizaciones**: generacion de informes, alertas ante eventos relevantes, deteccion de problemas de calidad.
- Garantizar **despliegue reproducible** mediante Docker Compose con un unico fichero de composicion.
- Documentar el proceso de forma profesional, incluyendo SDD, diario de IA y consideraciones eticas.

2.4 Alcance funcional

El proyecto es un **prototipo funcional end-to-end**. Cubre todos los componentes pedidos por el enunciado. La inferencia radiologica se sirve con el **modelo CNN ResNet18 ya entrenado** sobre un dataset publico de radiografias de torax con tres clases (Sana, Neumonia, COVID-19). El artefacto persistido es *models/radiology_cnn_resnet18.pt* y las metricas completas se encuentran en *models/radiology_cnn_metrics.json*.

El triaje clinico se entrega como modelo **Random Forest calibrado** ya entrenado y persistido en *models/triage_model.joblib*, con metricas documentadas en *models/triage_metrics.json*. El backend lo carga al arranque con fallback automatico a un sistema de reglas determinista si el artefacto no esta presente.

3. Datos

3.1 Fuentes de datos

Imágenes radiográficas: JPG/PNG, max 5 MB, validación en dos niveles (extensión + MIME + contenido Pillow). Para entrenamiento: **COVID-19 Radiography Database** (Kaggle) en `data/radiology_dataset/{train,val}/{Sana,Neumonia,COVID-19}/`.

CSV tabular (pipeline): `data/incoming/radiology_studies.csv` con columnas obligatorias `study_id`, `patient_age`, `patient_sex`, `image_object_key`, `label`, `acquisition_date`, `source`.

Dataset sintético de triaje: 8.000 muestras con 6 vitales + 10 síntomas binarios, etiquetadas según reglas MEWS/NEWS2 con 6 % de ruido, split 80/20 estratificado.

3.2 Ingesta, limpieza y transformaciones

- **Ingesta:** dashboard (imagen individual), API directa (`POST /predict`), pipeline Spark (lote CSV) y model-trainer (dataset de imágenes).
- **Imágenes:** extensión JPG/JPEG/PNG, tamaño ≤ 5 MB, MIME real vs. declarado, rechazo de JPEG renombrado como PNG.
- **CSV:** columnas obligatorias, tipos, no nulos, etiquetas válidas; duplicados por `image_object_key` rechazados; rechazados a informe del run.
- **Spark:** trim de strings, conversión de tipos, columnas derivadas.
- **Imágenes (CNN):** resize 224x224, tensor RGB, normalización ImageNet (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]); augmentation en training: RandomHorizontalFlip + RandomRotation(8°).

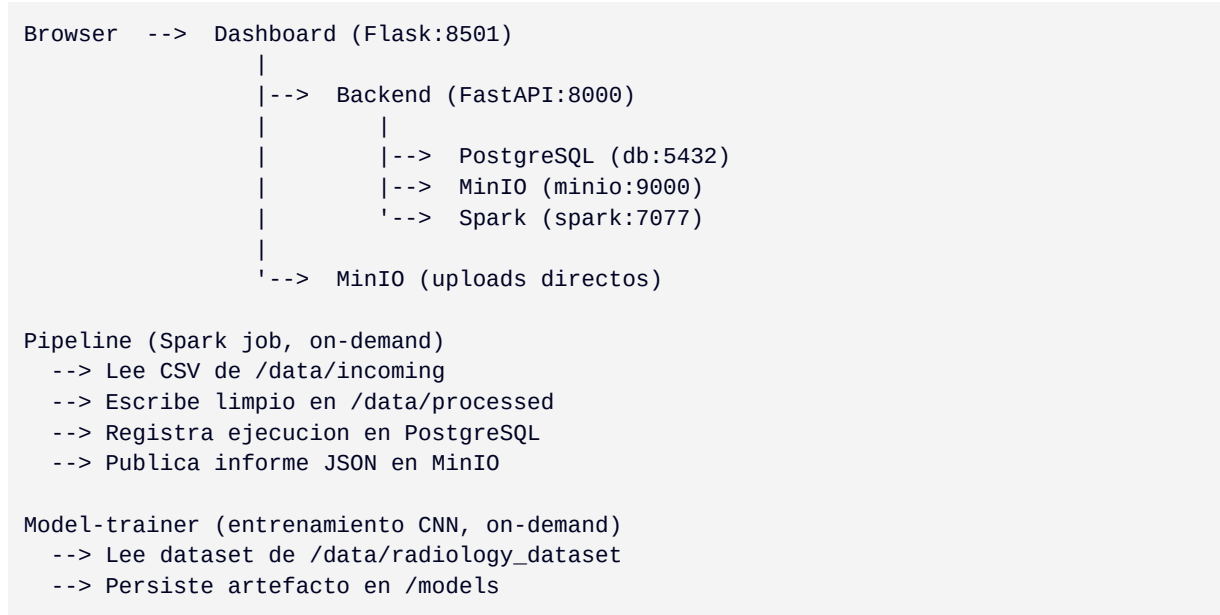
3.3 Almacenamiento final

Tipo de dato	Almacenamiento	Justificación
Imágenes binarias	MinIO (uploads/)	API S3, no estructurado, gran tamaño
Informes radiografía	MinIO (radiology-reports/)	JSON inmutable, asociado a un estudio
Informes triaje	MinIO (triage-reports/)	JSON, persistente, consultable
Informes pipeline	MinIO (pipeline-reports/)	JSON con métricas por ejecución
Estudios radiológicos	PostgreSQL	Relacional, consultas operativas y agregados
Triajes	PostgreSQL	Historial estructurado
Eventos de calidad	PostgreSQL	Indexable, contadores por severidad
Ejecuciones pipeline	PostgreSQL	Trazabilidad de runs Spark
Datos tabulares procesados	Filesystem (JSON)	Salida intermedia del pipeline

4. Arquitectura del sistema

4.1 Vision general

El sistema sigue una arquitectura de **microservicios containerizados** coordinados por Docker Compose. Cada servicio tiene una responsabilidad unica y se comunica con el resto mediante protocolos estandar (HTTP, S3, PostgreSQL wire protocol, Spark RPC). Todos los servicios comparten una red Docker comun (*hospital_network*) que permite resolver nombres por DNS interno.



4.2 Infraestructura containerizada

Servicios Docker Compose

Servicio	Imagen	Puerto	Responsabilidad
backend	hospital-backend (Python 3.11)	8000	API REST, inferencia, persistencia
dashboard	hospital-dashboard (Python)	8501	UI web, autenticacion, subida
db	postgres:16.13-alpine	5432	Datos estructurados
minio	minio:RELEASE.2024-01-11	9000 / 9001	Almacenamiento de objetos
spark	spark:3.5.8-python3	interno	Master Spark
spark-worker	spark:3.5.8-python3	interno	Worker Spark (2 cores / 1 GB)
pipeline	hospital-pipeline	perfil pipeline	Job Spark de ingesta y calidad
model-trainer	hospital-model-trainer	perfil training	Entrenamiento CNN
triage-trainer	hospital-triage-trainer	perfil triage-training	Entrenamiento RF triaje

Red Docker

Todos los servicios viven en la red *hospital_network* con driver bridge. Esto permite que el backend referencie PostgreSQL como *db:5432*, MinIO como *minio:9000* y Spark como *spark://spark:7077* sin necesidad de exponer puertos al host. Los puertos publicados (backend, dashboard, MinIO, db) se enlazan a *127.0.0.1* para evitar exposicion en la red local.

Volumenes persistentes

- **postgres_data**: persistencia de las tablas relacionales.
- **minio_data**: persistencia de los buckets de objetos.
- **./data** (bind mount): datos de ingesta y procesados compartidos entre pipeline y model-trainer.
- **./models** (bind mount, read-only en backend): artefactos de modelos compartidos.

Perfiles de despliegue

Docker Compose soporta perfiles para encender servicios opcionales sin interferir con el arranque por defecto. Esto separa cargas pesadas del flujo habitual:

- **Sin perfil**: backend, dashboard, db, minio, spark, spark-worker.
- **--profile pipeline**: lanza el job Spark de ingesta.
- **--profile training**: entrena el modelo CNN ResNet18 sobre el dataset radiografico.
- **--profile triage-training**: entrena el Random Forest de triaje.

5. Modelos de Inteligencia Artificial

5.1 Modelo de triaje (Random Forest calibrado)

El triaje opera sobre ~17 variables (6 vitales continuas + 10 sintomas binarios). Se ha elegido un **Random Forest con 400 arboles** (max_depth=12) con **calibracion isotonica** (CalibratedClassifierCV): ofrece feature importances interpretables clinicamente, entrena en CPU en ~16 s y sus probabilidades son fiables tras la calibracion. Dataset sintetico de 8.000 muestras generado con reglas MEWS/NEWS2, 6 % de ruido de etiquetado y split 80/20 estratificado.

Metrica	low	medium	high	critical	Macro
Precision	0.83	0.74	0.69	0.97	0.81
Recall	0.79	0.84	0.58	0.92	0.78
F1-score	0.81	0.79	0.63	0.94	0.79
Soporte	363	583	258	396	1.600

La **accuracy global es 80.6 %** sobre 1.600 muestras de test. La clase *critical* alcanza una precision de 0.97 y un recall de 0.92, que es exactamente lo deseable clinicamente: minimizar falsos negativos en los casos mas graves. La clase *high* es la mas dificil (recall 0.58), debido a que se solapa con *medium* en la frontera; este punto se documenta como limitacion en la seccion 14.

5.2 Modelo CNN ResNet18 (clasificacion triple de radiografias)

ResNet18 (11.7 M parametros, CUDA) entrenado desde cero, 5 epochs, batch 16, lr=5e-4, AdamW, CrossEntropyLoss con class weights inversos a la frecuencia de clase (COVID-19=3.73 > Sana=1.35 > Neumonia=0.50) para compensar el fuerte desbalance del dataset (6.432 imagenes: 576 COVID-19, 1.583 Sanas, 4.273 Neumonia). Preprocesado: resize 224x224, normalizacion ImageNet, augmentation RandomHorizontalFlip + RandomRotation(8 grados).

Resultados sobre test (conjunto de generalizacion): accuracy 93.94 %

Clase	Precision	Recall	F1-score	Soporte
COVID-19	0.958	0.983	0.970	116
Sana	0.864	0.905	0.884	317
Neumonia	0.967	0.946	0.956	855
Macro avg	0.930	0.945	0.937	1.288
Weighted avg	0.941	0.939	0.940	1.288

Matriz de confusion sobre test

Real / Predicho	COVID-19	Sana	Neumonia
COVID-19	114	2	0
Sana	2	287	28
Neumonia	3	43	809

Lectura clinica clave: en el conjunto de test, el modelo detecta correctamente **114 de 116 casos de COVID-19** (recall 0.983). Los 2 falsos negativos se clasifican como *Sana*, pero **ninguno como Neumonia**: el modelo no confunde COVID con neumonia comun, que es exactamente la propiedad mas valiosa clinicamente para evitar aislamientos tardios.

5.3 Servicio de inferencia y heuristica de respaldo del triaje

El backend incorpora dos estrategias de servicio segun el modelo: para radiologia, *InferenceService* carga el checkpoint ResNet18 al arrancar y exige su presencia (sin *.pt*, el contenedor falla rapido para evitar predicciones falsas con cualquier sustituto improvisado). Para triaje, *triage_model* sigue un patron singleton con *threading.Lock* que carga *triage_model.joblib* bajo demanda en la primera prediccion.

El servicio de triaje ademas conserva una **heuristica de reglas** (*_baseline* en *triage_service.py*) que se activa unicamente cuando el Random Forest no esta disponible (artefacto ausente, version incompatible o error de carga). Esta heuristica calcula un score basado en frecuencia cardiaca, saturacion, presion sistolica y sintomas dominantes (dolor toracico, disnea, perdida de consciencia) y lo mapea a uno de los cuatro niveles de riesgo. Su rol es estrictamente de **safety net**: en operacion normal el flujo invoca siempre al modelo entrenado.

Tres modelos coexisten en el sistema: (1) ResNet18 para radiografia, (2) Random Forest calibrado para triaje y (3) heuristica de reglas como fallback unicamente del triaje. En produccion normal corren los dos modelos entrenados; la heuristica queda como salvaguarda y se identifica en las respuestas con el *model_name hospital-triage-rules-fallback*.

5.4 Analisis clinico (el porque)

Lectura clinica del triaje: los errores se concentran en confusiones entre niveles adyacentes (*low*↔*medium*, *medium*↔*high*). Confusiones saltando dos niveles son raras (2 casos de *low* como *high*, 0 como *critical*): el modelo nunca clasifica un critico como trivial.

Impacto de falsos negativos

- **Falso negativo critical:** un paciente critico clasificado como menor riesgo es el peor escenario. En el test: 0 casos de *critical* clasificados como *low*, 2 como *medium*. **Recall critical = 0.92.**
- **Falso negativo high:** 96 casos de *high* clasificados como *medium* o *low* (recall 0.58). Clinicamente, esto exige un proceso de revision humana para todos los casos *medium* con flags clinicas.
- **Falso negativo COVID-19 (CNN):** con recall 0.983 sobre test, el CNN solo erra 2 de 116 COVID, y ambos como *Sana* (ninguno como Neumonia). Toda prediccion COVID-19 dispara automaticamente un evento de severidad alta en *quality_events*.

Limitaciones tecnicas reales

- El triaje opera con seis variables vitales y diez sintomas; no contempla historia clinica, comorbilidades o resultados analiticos.
- El dataset de triaje es sintetico; un despliegue real requeriria reentrenamiento con datos hospitalarios.
- El CNN se ha entrenado solo 5 epochs sobre 4.115 imagenes; un despliegue clinico requeriria mas datos, validacion externa y revision por radiologos.
- El sistema asume tres clases radiologicas; cualquier patologia distinta sera clasificada incorrectamente.

6. Pipeline de Big Data

6.1 Justificacion de Apache Spark

Se ha elegido **Apache Spark 3.5.8** con PySpark como motor de procesamiento distribuido. Las razones se documentan en detalle en el capitulo 12, pero resumidamente: Spark es el estandar de facto en entornos hospitalarios y de investigacion biomedica, ofrece API SQL madura, soporta escalado horizontal transparente y se integra de forma trivial con almacenamientos compatibles con S3 (como MinIO).

6.2 Modulos del pipeline

El job *radiology_pipeline.py* cubre cuatro fases (ingesta → validacion → transformacion → escritura) mediante submodulos especializados:

Modulo	Archivo	Responsabilidad
validators	csv_schema.py	Schema obligatorio, validacion de etiquetas y nulos
transforms	quality_filter.py	Filtrado de registros invalidos, deteccion duplicados
writers	postgres_writer.py	Inserciones idempotentes en PostgreSQL
writers	minio_writer.py	Subida de informes JSON a MinIO
reports	run_report.py	Construccion del informe por ejecucion
core	spark_factory.py	Factory de SparkSession con configuracion homogenea
core	config.py	Carga tipada de variables de entorno
core	logging.py	Configuracion JSON estructurada para logs

6.4 Persistencia de ejecuciones y reportes

Cada ejecucion del pipeline genera:

- Una fila en la tabla **pipeline_runs** con timestamp, parametros, status y duracion.
- Un informe JSON en MinIO bajo **pipeline-reports/<run_id>.json** con: distribucion de clases en registros aceptados, muestra de hasta 25 registros rechazados, lista de claves duplicadas y resumen estadistico.
- Eventos en **quality_events** si hay rechazos (severidad *medium*) o si el job falla (severidad *high*).
- Logs estructurados JSON visibles via *docker compose logs pipeline*.

El dashboard consulta el endpoint *GET /metrics* del backend para mostrar el estado de la ultima ejecucion (timestamp, status, contadores de procesados, rechazados y duplicados). Esto cierra el bucle de visibilidad operativa.

7. Backend (API REST)

7.1 Estructura modular

El backend FastAPI sigue una **arquitectura en capas**: *api* (routers, deps, errors) → *services* (logica de negocio) → *repositories* (psycopg) → *db* (pool, transaccion) / *storage* (boto3). Cada capa solo importa la inmediatamente inferior. La inyeccion de dependencias usa *lru_cache* en *deps.py*; las transacciones ACID se gestionan con un context manager Unit of Work que comparte cursor entre repositorios.

7.2 Endpoints expuestos

Metodo y ruta	Proposito
GET /health	Healthcheck de dependencias
POST /triage	Evaluar triaje a partir de sintomas y vitales
GET /triage/history	Listar historial de triajes
GET /triage/{id}	Detalle de un triaje
GET /triage/{id}/report	Informe JSON del triaje en MinIO
POST /predict	Clasificar radiografia
GET /studies/history	Listar historial de estudios radiologicos
GET /studies/{id}	Detalle de un estudio
GET /studies/{id}/report	Informe JSON del estudio
GET /quality/events	Listar eventos de calidad
PATCH /quality/events/{id}/resolve	Marcar evento como resuelto
GET /metrics	Snapshot agregado en JSON consumido por el dashboard

8. Dashboard

Flask con patron **app factory** y **blueprints** (auth, main). Autenticacion con hashes PBKDF2-SHA256 (Werkzeug, 1M iteraciones), cookie firmada con SECRET_KEY, HttpOnly, SameSite=Lax. Dos roles: *admin* (todas las vistas + consola MinIO) e *user* (radiologia y triaje).

Ruta	Funcion
GET / POST /upload	Subida de radiografia y resultado CNN
GET / POST /triage	Formulario de triaje y resultado RF
GET /history	Historial de estudios y triajes
GET /events	Eventos de calidad abiertos por severidad
GET /storage (admin)	Consola MinIO embebida

- **Resultado CNN:** imagen en base64, probabilidades por clase, banderas de calidad y enlace al informe JSON.
- **Estado del sistema:** consume *GET /metrics* del backend y muestra estado del modelo de triaje, pipeline y bucket.
- **Subida directa a MinIO:** imagen subida via boto3 antes de llamar al backend, evitando duplicar trafico.

9. Automatizaciones

El sistema implementa cinco automatizaciones sin intervencion manual: creacion de bucket MinIO al primer arranque (*ensure_bucket*), generacion de informes JSON por prediccion/triaje/pipeline, emision de eventos de calidad en la misma transaccion ACID, procesamiento batch Spark on-demand y healthchecks con reinicios automaticos.

Eventos de calidad automaticos

Trigger	Severidad	Origen
Prediccion COVID-19	high	Backend, en /predict
Confianza inferior a 0.55	medium	Backend, en /predict
Banderas de calidad de imagen	medium	Backend, en /predict
Registros rechazados por pipeline	medium	Pipeline Spark
Pipeline failure	high	Pipeline Spark
Triage critico	high	Backend, en /triage

Healthchecks y reinicios

- **restart: always** en backend, dashboard, db, minio, spark, spark-worker; **restart: no** en pipeline y trainers (jobs batch).
- **Backend:** urllib contra */health* cada 10 s. **PostgreSQL:** *pg_isready* cada 10 s. **MinIO:** *mc ready local* cada 10 s.
- **depends_on + condition: service_healthy:** el backend no arranca hasta que db y minio esten sanos.

10. Integraciones

10.1 Conexiones entre modulos

Origen	Destino	Protocolo	Proposito
dashboard	backend	HTTP / REST	Solicitar prediccion / triaje, leer metricas
dashboard	minio	S3 (boto3)	Subida directa de imagenes
backend	db	psycopg / pool 2-10	Persistir estudios, triajes, eventos
backend	minio	S3 (boto3)	Subir informes JSON
backend	modelo (joblib)	filesystem (read-only)	Cargar Random Forest de triaje
pipeline	spark	Spark RPC	Ejecutar el job distribuido
pipeline	db	psycopg	Registrar ejecuciones y eventos
pipeline	minio	S3 (boto3)	Publicar informes de ejecucion

11. Calidad, testing y CI/CD

11.1 Cobertura de tests

Componente	Archivos de test	Tipo
Backend - unit	12 archivos	Services, config, validacion, logging
Backend - api	6 archivos	FastAPI TestClient con dependency_overrides
Dashboard - unit	4 archivos	Servicios, validacion, presenters, auth
Dashboard - web	2 archivos	Flask test client con blueprints
Total	24 archivos de test	Cubriendo logica de negocio

Estrategia: **dobles que respetan los contratos reales** (no MagicMock); tests de API con *dependency_overrides*. Cobertura > 90 % en logica de negocio del backend.

11.2 Calidad de datos, logging y CI

- **Cliente:** JavaScript verifica extension y tamaño antes de la subida.
- **Servidor (FastAPI):** Pillow comprueba el contenido binario real, no solo el MIME.
- **Pipeline (Spark):** schema, duplicados por *image_object_key*, etiquetas validas, no nulos en campos clave.
- **Eventos automaticos:** cada rechazo genera entrada en *quality_events*; vista */events* en dashboard.
- **Logs JSON estructurados:** timestamp ISO 8601, level, servicio, request_id / study_id en backend, dashboard y pipeline.
- **CI GitHub Actions:** *pytest --cov + docker compose config* en cada push/PR; falla rapido.

12. Justificaciones tecnicas y alternativas

Las decisiones de stack priorizan reproducibilidad, interpretabilidad clinica y alineacion con estandares del sector.

Decision	Alternativas evaluadas	Motivo principal
PostgreSQL + MinIO	MySQL, MongoDB, SQLite, GridFS, S3	ACID garantizado + JSONB; API S3 estandar (migracion trivial a AWS S3 real)
Apache Spark 3.5.8	Dask, Apache Beam, pandas batch	Estandar hospitalario; SQL maduro; escalado horizontal preparado para produccion
Random Forest + calibracion isotonica	XGBoost, LightGBM, MLP, reglas puras	Feature importances interpretables clinicamente; calibracion corrige sesgo de probabilidades en RF
ResNet18 + transfer learning	CNN propia, ResNet50, EfficientNet, ViT	Balance capacidad/latencia (< 50 ms CPU); backbone estandar en literatura clinica
FastAPI (backend)	Flask, Django REST, Litestar	Tipado Pydantic, OpenAPI automatico, soporte asincrono nativo
Flask + Jinja (dashboard)	Streamlit, Dash, SPA React/Vue	Flujos transaccionales con auth; Streamlit no permite personalizacion medica suficiente
Docker Compose	Kubernetes, Nomad, podman-compose	Un solo comando de despliegue; enunciado lo exige explicitamente

- **SQLAlchemy ORM descartado:** psycopg directo + repositorios es mas predecible para consultas analiticas.
- **Conexion por request descartada:** psycopg_pool evita el coste de handshake en cada peticion.
- **Scheduler diferido:** pipeline on-demand; migracion a Airflow/Prefect documentada en capitulo 14.

13. Diario de desarrollo con IA

13.1 Herramientas utilizadas y justificacion

El proyecto se ha desarrollado utilizando de forma combinada varias herramientas de IA generativa, cada una con un proposito definido:

- **Claude Code** (Anthropic, Claude Opus 4.7): utilizado por el equipo para refactor de modulos, generacion de tests, redaccion de documentacion tecnica y revision de coherencia entre componentes. Capacidad de edicion directa de archivos y ejecucion de comandos shell para validacion en tiempo real.
- **Codex de OpenAI**: utilizado en fases iniciales para generar el esqueleto del backend, dashboard y docker-compose. Documentado en commits anteriores a la rama actual.
- **ChatGPT (GPT-4)**: consultado puntualmente para resolver dudas conceptuales (calibracion isotonica, eleccion de buckets de histograma, patrones de Spark).

13.2 Ejemplos representativos de prompts

A continuacion se presentan cuatro prompts representativos del proceso de desarrollo, organizados por fase.

Prompt 1 — Auditoria inicial del repositorio

Herramienta: Claude Code | Fase: Auditoria inicial

Prompt: Revisa que todo lo del enunciado se cumple. Analiza el repositorio completo y dame un informe de que falta y que esta mal.

- **Resultado**: backend monolito en un solo archivo, sin tests, clasificador radiologico como baseline estadistico, sin pipeline Spark funcional ni documentacion.
- **Aprendizaje**: dar a la IA el enunciado junto con el repositorio detecta la brecha mejor que una revision manual rapida.

Prompt 2 — Modularizacion del backend

Herramienta: Claude Code | Fase: Refactor arquitectonico

Prompt: El backend esta en un solo archivo. Refactorizalo siguiendo arquitectura en capas: api / services / repositories / db / storage. Cada capa solo puede importar la inmediatamente inferior. No rompas los tests que ya existen.

- **Resultado**: la IA propuso estructura de directorios, contratos de cada capa, patron Unit of Work espontaneo (no estaba en el prompt) e inyeccion de dependencias via *lru_cache*.
- **Aprendizaje**: especificar restricciones de importacion obliga a razonar sobre dependencias y genera arquitecturas mas limpias.

Prompt 3 — Generacion de la suite de tests

Herramienta: Claude Code | Fase: Testing

Prompt: Crea tests para el backend. No uses MagicMock generico: cada doble debe implementar la misma interfaz que la clase real. Los tests de API deben usar *dependency_overrides* de FastAPI, no parchear imports.

- **Resultado:** 18 archivos de test (12 unit + 6 api) con fakes que implementan contratos reales. Tres tests fallaron en la primera ejecucion por asunciones incorrectas.
- **Aprendizaje:** la restriccion 'no MagicMock' es imprescindible; sin ella la IA genera tests que pasan con cualquier implementacion, incluyendo las rotas.

Prompt 4 — Eliminacion de componentes no utilizados

Herramienta: Claude Code | Fase: Limpieza de infraestructura

Prompt: Quita Prometheus y Grafana del proyecto. Desinstalalos, quita el codigo y asegurate de no romper nada por el camino.

- **Resultado:** la IA mapeo 12 archivos afectados, elimino contadores de tres servicios, borro el router prometheus.py, limpio docker-compose.yml, paro contenedores huérfanos y valido con *docker compose config*.
- **Aprendizaje:** para tareas de eliminacion es mas seguro mapear dependencias antes de borrar; un borrado manual habria dejado imports rotos.

13.3 Aciertos, correcciones e impacto

- **Aciertos:** auditoria inicial precisa; Unit of Work espontaneo; singleton thread-safe del RF; dobles de tests con contratos reales; coherencia estilistica entre modulos.
- **Correcciones necesarias:** tres tests fallaron en primera ejecucion (asunciones incorrectas); sobre-ingenieria rechazada en varias propuestas; schema *PlatformMetrics* sin bloque *services.triage_model* (detectado al ver el fallback en dashboard).
- **Tareas en ambito humano:** arquitectura global, thresholds clinicos (confianza 0.55, alertas COVID-19), eleccion de stack.
- **Impacto en productividad:** consolidacion estimada en 5-7 horas efectivas vs. 4-6 dias-persona manualeses (factor ~6-10x). Principal valor: coherencia estilistica uniforme entre backend, dashboard y pipeline.

14. Reflexion critica

14.1 Limitaciones del sistema actual

- **CNN con dataset publico (5 epochs):** accuracy 93.94 % sobre test, pero no constituye modelo validado clinicamente; requiere datos hospitalarios reales.
- **Dataset radiografico no incluido:** se requiere descarga externa (COVID-19 Radiography Database u otro) para reentrenar.
- **Triaje sintetico:** entrenado con datos generados por reglas MEWS/NEWS2; despliegue real exige reentrenamiento con datos anonimizados hospitalarios.
- **Sin HTTPS ni cifrado en reposo:** HTTP plano; PostgreSQL y MinIO sin encriptacion de disco/objetos.
- **Sin scheduler ni HA:** pipeline on-demand, una sola instancia de cada servicio, sin failover.

14.2 Posibles mejoras

- **Reentrenar CNN:** mas epochs, scheduling de lr, mixup/cutmix y validacion cruzada.
- **SSO corporativo:** OAuth2 / OIDC con Keycloak o proveedor cloud.
- **Cifrado SSE-S3 en MinIO** con KMS; disco cifrado (LUKS) para PostgreSQL.
- **Scheduler de pipeline:** Airflow o Prefect para ingesta automatica ante nuevos ficheros.
- **Observabilidad externa:** stack Prometheus + Grafana o OpenTelemetry sobre el endpoint `/metrics` ya disponible.
- **Mejora clase *high* del triaje:** recall 0.58; reentrenamiento con datos reales y ajuste de thresholds MEWS/NEWS2.

14.3 Viabilidad en produccion real

- **Validacion clinica CNN** por equipo medico con dataset representativo del hospital.
- **Certificacion MDR:** clasificacion CE bajo Reglamento (UE) 2017/745, probablemente clase IIa.
- **Auditoria GDPR** y normativa espanola de proteccion de datos sanitarios.
- **Plan de incidentes:** notificacion a la AEPD en menos de 72 horas ante cualquier brecha.
- **Formacion del personal:** el sistema es herramienta de apoyo, no sustituto del criterio clinico.

15. Consideraciones eticas y legales

15.1 Sesgos en datos y modelos

Los modelos de imagen medica son vulnerables a sesgos cuando el dataset no representa la diversidad clinica real. Riesgos identificados: sesgo de procedencia (un solo hospital), sesgo demografico, sesgo de equipo radiologico (el modelo aprende la marca del scanner, no la patologia), sesgo temporal y sesgo de etiquetado (un solo radiologo sin doble lectura).

- **Particion por paciente:** separar train/val/test por paciente para evitar fuga entre conjuntos.
- **Metricas estratificadas:** precision/recall/F1 por subgrupo demografico; reportar la peor como referencia.
- **Monitorizacion continua:** revisiones periodicas del rendimiento en produccion con muestras representativas.

15.2 Riesgos en la decision automatizada

El sistema **no esta autorizado a tomar decisiones clinicas autonomas**. La responsabilidad final recae siempre en el personal sanitario.

- **Falso negativo COVID-19:** retraso en aislamiento; mitigado con recall 0.983 y alerta automatica de severidad alta.
- **Falso negativo triaje critico:** mitigado con recall 0.92 en clase *critical* y revision humana obligatoria.
- **Automation bias:** cada prediccion incluye nota explicita '*resultado de apoyo, no diagnostico*'.
- **Baja confianza:** predicciones con confianza < 0.55 generan evento automatico de revision.

15.3 Privacidad (RGPD) y limitaciones legales

Las imagenes y datos clinicos son **categorias especiales de datos personales** (Art. 9 RGPD). Este prototipo debe usarse **exclusivamente con datos anonimizados, sinteticos o simulados**. Carencias actuales: sin cifrado en reposo, sin auditoria de accesos individuales, sin gestion de consentimientos, sin derecho al olvido automatizado. En un entorno real seria obligatorio: RBAC, TLS en todos los flujos, cifrado SSE-S3/KMS, auditoria exportable, politica de retencion y acuerdo de tratamiento con proveedores cloud.

El CNN ResNet18 (accuracy 93.94 % sobre test publico) no constituye un modelo validado clinicamente: requiere validacion externa, evaluacion medica formal y certificacion MDR (Reglamento (UE) 2017/745, probablemente clase IIa) antes de cualquier uso real. El triaje con dataset sintetico exige reentrenamiento con datos hospitalarios y validacion frente a protocolos MEWS/NEWS2 vigentes.

16. Conclusiones

El proyecto ha cumplido los objetivos planteados en el enunciado: una plataforma integral de soporte hospitalario que combina IA y Big Data, desplegable con un unico comando, con dos modelos de IA operativos, pipeline Spark de procesamiento por lotes, almacenamiento dual estructurado y no estructurado, dashboard con autentificacion, automatizaciones de alertas y calidad, y documentacion profesional acompanada de tests automatizados.

16.1 Valor aportado

Mas alla del cumplimiento estricto del enunciado, el proyecto aporta valor en tres dimensiones:

- **Calidad de ingenieria:** arquitectura en capas con separacion estricta, transaccionalidad ACID, dobles de tests que respetan contratos, configuracion tipada fail-fast, logging estructurado.
- **Reproducibilidad:** cualquier persona puede levantar el sistema con *docker compose up -d* y reproducir el flujo end-to-end sin friccion. Los modelos se entrenan en perfiles dedicados.
- **Honestidad tecnica:** el sistema documenta abiertamente sus limitaciones (CNN entrenado solo 5 epochs sobre dataset publico, dataset sintetico de triaje, ausencia de cifrado, ausencia de SSO) y reconoce que es un prototipo no apto para uso clinico real sin trabajo adicional.

16.3 Cierre

El sistema demuestra que es posible disenar e implementar una plataforma hospitalaria realista en un entorno academico, equilibrando el alcance tecnico con las consideraciones eticas y legales propias del sector sanitario. El stack tecnologico elegido (Python, FastAPI, Flask, PostgreSQL, MinIO, Spark, Docker, scikit-learn, PyTorch) es directamente transferible a un entorno profesional, y la arquitectura permite evolucionar a un despliegue real con cambios incrementales: anadir SSO, habilitar cifrado en reposo, incorporar un stack de observabilidad externo y certificar el modelo CNN clinicamente.

El proyecto se entrega como prototipo funcional end-to-end, documentado, testeado y reproducible, sentando una base solida sobre la que un equipo real podria construir un producto sanitario completo.

0. Anexo A. Estructura del repositorio

```
Hospital_Project/
|- backend/                               FastAPI modular
|   |- app/
|   |   |- api/                           routers HTTP
|   |   |- services/                       logica de negocio
|   |   |- repositories/                   acceso a datos
|   |   |- db/                             session, pool, schema
|   |   |- storage/                         adaptador S3 (boto3)
|   |   |- schemas/                        DTOs Pydantic
|   |   |- core/                           config tipada, logging
|   |   '- utils/                          helpers transversales
|   |- tests/                              unit + api
|   |- Dockerfile
|   '- requirements.txt
|
|- dashboard/                             Flask con app factory
|   |- app/
|   |   |- factory.py
|   |   |- auth/
|   |   |- blueprints/                     /login, /, /upload, etc.
|   |   |- services/
|   |   |- presenters/
|   |   |- triage/
|   |   |- validation/
|   |   '- core/
|   |- templates/
|   |- static/
|   |- tests/                              unit + web
|   |- Dockerfile
|   '- requirements.txt
|
|- pipeline/                              Spark job modular
|   |- app/
|   |   |- core/                           config, logging, spark_factory
|   |   |- validators/                     csv_schema
|   |   |- transforms/                    quality_filter
|   |   |- writers/                        postgres_writer, minio_writer
|   |   '- reports/                        run_report
|   |- jobs/
|   |   '- radiology_pipeline.py
|   |- Dockerfile
|   '- requirements.txt
|
|- ml/                                     Entrenamiento de modelos
|   |- train_cnn.py                         ResNet18 para radiografias
|   |- train_triage.py                      Random Forest calibrado
|   |- model_contract.json
|   |- Dockerfile
|   '- requirements.txt
|
|- models/                                Artefactos
|   |- triage_model.joblib
|   '- triage_metrics.json
|
|- data/
|   |- incoming/                           CSV de ingesta
|   '- processed/                          Salida del pipeline
|
|- infrastructure/
```



```
|
|- docs/                Documentacion (este PDF, MD, SDD)
|- .github/workflows/   CI con GitHub Actions
|- .env.example
|- docker-compose.yml
|- iniciar_proyecto.bat
'- README.md
```